Deterministic ML-KEM/ML-DSA Handshake for Quantum-Safe Peer Messaging

Shivansh Agrawal
Department of Information Technology,
Indian Institute of Information Technology Sonepat
Sonepat, India
shivansh.agrprof@gmail.com

Aditya Kumar
Department of Computer Science and Engineering,
Indian Institute of Information Technology Sonepat
Sonepat, India
shivansh.agrprof@gmail.com

***Abstract*—This paper presents a prototype implementation of a post-quantum secure peer messaging system built with ML-KEM-768 for key encapsulation, ML-DSA-44 for signature-based identity operations, and AES-GCM-256 for message confidentiality and integrity. The system is implemented in Python with FastAPI transport and liboqs cryptographic primitives on Windows. A deterministic seed derivation workflow is integrated to enable repeatable key recovery for decapsulation in a constrained two-party handshake design. We report implementation-grounded observations from runtime network traces, including artifact sizes, handshake timing, and encrypted payload overhead. The measured baseline handshake completed in 367 ms on a local network, and encrypted chat delivery round-trip times were 204 ms and 1236 ms for two observed transmissions. We discuss the cybersecurity implications of deterministic seed reuse, global shared-secret lifecycle, and endpoint trust assumptions. The work is positioned as a research prototype and deployment study, with a benchmark expansion plan for future large-scale evaluation.**

***Index Terms*—post-quantum cryptography, ML-KEM, ML-DSA, liboqs, secure messaging, FastAPI, cybersecurity**

I. Introduction

Large-scale quantum attacks on classical public-key primitives motivate cryptographic migration planning in operational systems [10]. The transition to post-quantum cryptography (PQC) has become a practical engineering priority as standards and software stacks mature. NIST has finalized FIPS standards for key encapsulation and digital signatures in the ML-KEM and ML-DSA families [1], [2], creating a path for real-world systems to move beyond theoretical evaluations.

This paper documents the design and implementation of a quantum-safe peer messaging prototype that combines ML-KEM-768, ML-DSA-44, and AES-GCM-256. The implementation objective is not to claim production hardening; instead, it is to demonstrate a complete end-to-end handshake and encrypted communication workflow in a developer-operational environment (Windows, Python, FastAPI, liboqs).

The contribution is threefold:

- A deterministic handshake workflow that allows reproducible decapsulation key recovery from signature-derived seed material.
- A deployable two-peer network service with automatic handshake, encapsulation callback, and encrypted message exchange.
- A trace-driven evaluation snapshot with reproducible numeric data suitable for publication tables and graphing.

II. Background and Context

liboqs is a widely used open-source library for experimentation and prototyping in PQC and tracks NIST standard algorithms, including ML-KEM and ML-DSA [4]. The project also explicitly warns that it is intended for research use and not yet recommended for production protection of sensitive data [5]. This caveat directly influences how claims are framed in this manuscript.

In the presented system, ML-KEM-768 is used for shared secret establishment and ML-DSA-44 is used to bind protocol material to an identity key. AES-GCM provides authenticated encryption for chat messages once a shared secret is established. A Drand endpoint contributes public randomness to the seed derivation stage [7].

III. Related Work and Positioning

Recent post-quantum engineering literature emphasizes migration pathways, hybrid deployment, and implementation hardening rather than algorithm novelty alone. This work aligns with that systems-oriented direction by focusing on operational integration of standards-track primitives into a networked messaging stack. In contrast to papers centered on cycle-level benchmarking only, this manuscript prioritizes protocol assembly, endpoint behavior, and attack-surface trans- parency under realistic developer deployment constraints.

IV. System and Threat Model

A. System Model

The prototype consists of two peers running the same FastAPI service. One peer starts in INITIATOR role and the second in LISTENER role. The INITIATOR sends a handshake offer containing identity and KEM public artifacts. The LISTENER encapsulates to the received KEM key and replies with ciphertext for decapsulation.

B. Threat Model Assumptions

The implementation assumes:

- A non-malicious local deployment environment for prototype evaluation.
- Network-level visibility by potential adversaries, requiring confidentiality and integrity protection.
- No dedicated hardware root of trust or secure enclave.

The implementation does *not* currently provide formal authenticated key exchange guarantees under active man-in-the-middle analysis. This limitation is addressed in Section VII.

V. Design and Implementation

A. Handshake Workflow

The cryptographic sequence is implemented across the service and helper modules:

1) Generate ML-DSA-44 identity keypair.
2) Fetch Drand randomness and sign it.
3) Derive deterministic seed using SHAKE-256 over the signature output.
4) Switch liboqs randomness callback to custom deterministic mode.
5) Generate ML-KEM-768 keypair and send public artifacts to remote peer.
6) Remote peer encapsulates and returns KEM ciphertext.
7) Initiator re-derives deterministic state and decapsulates.
8) Both peers use resulting shared secret for AES-GCM chat encryption.

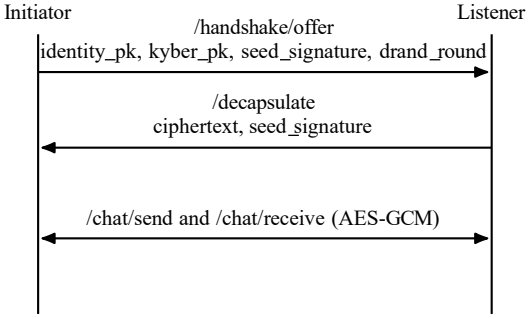

Fig. 1. Protocol flow for deterministic ML-KEM/ML-DSA handshake and encrypted chat.

B. Implementation Notes

The implementation uses Python, FastAPI [8], requests, cryptography (AESGCM), and liboqs-python bindings. On Windows, the liboqs shared library is loaded via an environment-configured DLL path. A global lock is used to prevent race conditions because random callback state is process-global during deterministic operations.

C. Message Protection

AES-GCM uses a 12-byte nonce and appends a 16-byte authentication tag to ciphertext. For plaintext length P bytes, cryptographic payload size excluding JSON serialization is:

$$S_{enc} = 12 + (P + 16) = P + 28 \quad (1)$$

This fixed 28-byte additive overhead has significant relative impact on very short messages.

VI. Experimental Snapshot from Runtime Traces

This section reports values observed in runtime logs captured during prototype execution. These values represent implementation evidence, not hardware-normalized benchmark claims.

A. Observed Cryptographic Artifact Sizes

TABLE I
Observed Artifact Sizes from Handshake Trace

Artifact	Size (bytes)
ML-DSA-44 identity public key	1312
ML-KEM-768 public key	1184
ML-DSA-44 seed signature	2420
ML-DSA-44 certification signature	2420
ML-KEM-768 ciphertext	1088
Drand randomness sample	32

B. Observed Handshake Latency

TABLE II
Observed Handshake Timing (ms)

Round	Offer-¿Decap	Decap-¿Ack	Total
1	227	140	367
2	5309	2329	7638

Round 1 represents a low-latency successful exchange. Round 2 demonstrates a delayed exchange under runtime variability and retry behavior.

C. Observed Encrypted Chat Transport Cost

VII. Security Discussion

The prototype demonstrates an operational post-quantum messaging flow, but several security-relevant aspects require caution:

- **Deterministic recovery tradeoff:** signature-derived deterministic seed reuse can reduce entropy diversity and

TABLE III
OBSERVED CHAT ENCRYPTION OVERHEAD

Message	Plaintext (bytes)	Nonce (bytes)	Ciphertext+Tag (bytes)	Total Enc. Payload (bytes)	RTT (ms)
”yo”	2	12	18	30	204
”yoo”	3	12	19	31	1236

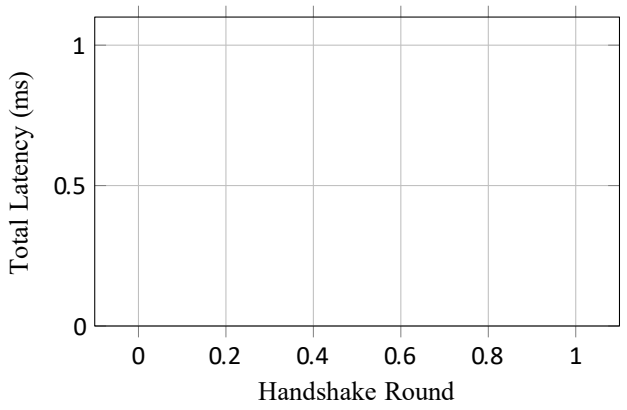

Fig. 2. Observed handshake total latency from runtime traces.

should be replaced with stronger key schedule design for production.

- **Global shared secret lifecycle:** shared secret is kept in process memory and reused for session traffic; forward secrecy properties are limited.
- **Endpoint trust assumptions:** identity and offer handling is implementation-level and does not yet represent a complete formal authenticated key exchange proof.
- **Missing explicit peer verification step:** the current listener path encapsulates to a received public key but does not perform full protocol-level identity validation before session acceptance.
- **Prototype scope warning:** liboqs itself is published with explicit research-prototype guidance [5], [9].

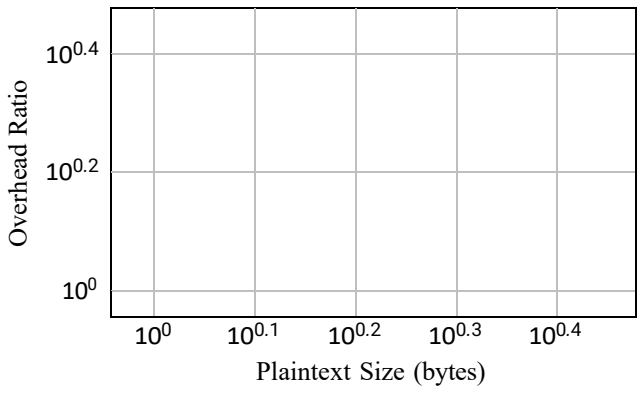

Fig. 3. Encryption overhead ratio versus plaintext size (log-log scale).

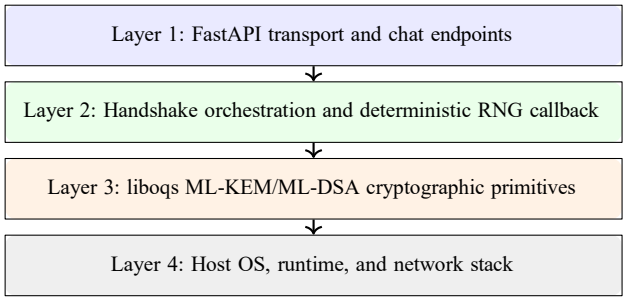

Fig. 4. System architecture layers for implementation decomposition.

VIII. LIMITATIONS AND FUTURE WORK

A. Current Limitations

- Evaluation data currently comes from runtime traces, not large-sample controlled benchmarks.
- Testing context is two-peer LAN execution; internet-scale, packet-loss, and adversarial scenarios are not yet covered.
- No formal proof of active attack resistance for the complete transport protocol.

B. Planned Benchmark Extension

Future work will run standardized performance and memory suites from liboqs test tooling, then compare:

- ML-KEM and ML-DSA operation latency distributions across 1000+ samples.
- Throughput under concurrent sessions.
- Memory footprint and CPU utilization under controlled load.
- Comparative profile against a hybrid classical plus PQ fallback mode.

IX. REPRODUCIBLE GRAPH DATA (CSV)

The following CSV files are included for direct graph generation:

- handshake_latency_ms.csv
- protocol_artifact_sizes.csv
- chat_rtt_ms.csv
- message_overhead_profile.csv

These datasets are consumed directly by the chart figures in this manuscript using PGFPlots.

X. CONCLUSION

This paper presented a complete implementation study of deterministic ML-KEM/ML-DSA handshake integration for post-quantum peer messaging. The prototype demonstrates successful key establishment and encrypted communication using standards-aligned algorithms and mainstream service tooling. Runtime trace analysis provides concrete artifact and latency evidence suitable for an IEEE implementation paper. The security analysis clarifies why the current design is an engineering prototype rather than a production profile and outlines a benchmark-driven roadmap for the next evaluation stage.

REFERENCES

[1] NIST, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard," National Institute of Standards and Technology, 2024.

[2] NIST, "FIPS 204: Module-Lattice-Based Digital Signature Standard," National Institute of Standards and Technology, 2024.

[3] NIST, "FIPS 205: Stateless Hash-Based Digital Signature Standard," National Institute of Standards and Technology, 2024.

[4] Open Quantum Safe, "liboqs: Open-Source C Library for Post-Quantum Cryptography," GitHub repository. [Online]. Available: https://github.com/open-quantum-safe/liboqs

[5] Open Quantum Safe, "liboqs README: Limitations and Security," 2025. [Online]. Available: https://github.com/open-quantum-safe/liboqs/blob/main/README.md

[6] NIST, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)," NIST SP 800-38D, 2007.

[7] drand, "Distributed Randomness Beacon Project," [Online]. Available: https://drand.love

[8] FastAPI, "FastAPI Documentation," [Online]. Available: https://fastapi.tiangolo.com

[9] Open Quantum Safe, "liboqs SECURITY.md," 2025. [Online]. Available: https://github.com/open-quantum-safe/liboqs/blob/main/SECURITY.md

[10] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proc. 35th Annual Symposium on Foundations of Computer Science*, 1994, pp. 124–134.